

My Movie Recommender

Anirudh S (Roll Number: 23035010660)

Term Project Report

Trimester VII

B.Sc. (Honours) Data Science and Artificial Intelligence

Indian Institute of Technology Guwahati, India

anirudh.s@op.iitg.ac.in

ABSTRACT

Recommendation systems are extensively used in modern entertainment platforms to assist users in discovering relevant digital content efficiently. This project presents the development of a lightweight content-based movie recommendation system using Natural Language Processing (NLP) and cosine similarity techniques.

The implemented recommendation system analyzes descriptive movie tags such as genres, themes, and keywords to identify similar movies. A curated dataset containing movie titles, tags, and poster image URLs was used during experimentation. Count Vectorization was applied to convert textual movie descriptions into numerical vectors suitable for similarity analysis. Cosine similarity was then used to compute similarity scores between movies.

The recommendation engine was integrated into an interactive frontend application developed using Streamlit. Users can select a movie title from a dropdown menu and instantly obtain five recommended movies along with corresponding movie posters.

Experimental observations indicate that content-based filtering can generate meaningful recommendations without requiring user ratings or interaction history. The project demonstrates the practical implementation of machine learning and NLP techniques within recommendation systems using computationally efficient methods.

Index Terms— Movie Recommendation System, Natural Language Processing, Cosine Similarity, Streamlit, Machine Learning, Content-Based Filtering

1 INTRODUCTION

Recommendation systems have become an essential component of modern digital entertainment platforms such as Netflix, YouTube, and Spotify. These systems assist users in identifying relevant content from massive collections of movies, videos, and multimedia resources. Recommendation algorithms significantly improve user engagement by reducing the effort required for searching and selecting suitable content.

This project focuses on the implementation of a content-based movie recommendation system. Unlike collaborative filtering approaches that depend heavily on user interaction history and ratings, content-based filtering recommends movies using information directly associated with the

movies themselves. This approach is especially beneficial for handling the cold start problem where newly registered users possess limited interaction data.

The developed recommendation system utilizes descriptive movie tags containing genres, themes, and important keywords. Natural Language Processing techniques were applied to preprocess textual information and transform movie descriptions into numerical vectors using Count Vectorization. Cosine similarity was then used to measure similarity between movie vectors.

The entire application was implemented using Python programming language with libraries such as Pandas, Scikit-learn, and Streamlit. The final system allows users to select a movie title and instantly receive movie recommendations together with corresponding poster images.

Section II discusses the problem statement and project objectives. Section III explains the methodology and implementation details. Experimental setup and results are presented in Section IV. Section V discusses observations and limitations, followed by conclusions and future work in Section VI.

2 PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Streaming platforms currently provide access to thousands of movies across multiple genres and languages. Due to the large amount of available content, users often experience difficulty identifying movies aligned with their interests. This challenge is commonly referred to as decision paralysis.

Traditional recommendation systems often rely on user watch history, ratings, and interaction behavior. However, newly registered users may not possess sufficient historical data, resulting in poor recommendation quality. This issue is known as the cold start problem.

The primary objective of this project was to design and implement a lightweight recommendation system capable of suggesting similar movies using movie content information instead of user interaction data.

2.2 Objectives

The major objectives of the project are listed below:

1. To study recommendation system techniques used in entertainment platforms.
2. To design and implement a content-based movie recommendation engine.
3. To preprocess textual movie tags using Natural Language Processing methods.
4. To apply Count Vectorization for converting text into numerical feature vectors.
5. To compute movie similarity using cosine similarity.
6. To develop an interactive frontend interface using Streamlit.
7. To display movie recommendations together with poster images.

3 METHODOLOGY / APPROACH

This section describes the methodology adopted for implementing the recommendation system.

3.1 Overall Workflow

The workflow of the implemented recommendation system is illustrated in Figure 1.

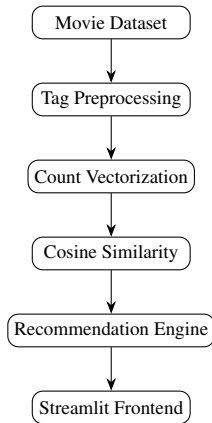


Fig. 1. Workflow of the content-based movie recommendation system.

The workflow begins by loading the movie dataset stored in CSV format. Each dataset entry contains movie identifiers, titles, descriptive tags, and poster image URLs. Text preprocessing operations are then applied before converting movie tags into numerical vectors using Count Vectorization.

Cosine similarity scores are calculated between movie vectors to identify relationships among movies. Finally, the recommendation engine selects the top five most similar movies and displays them through the Streamlit-based frontend application.

3.2 Technical Details

The project was implemented using Python due to its strong support for machine learning and data analysis applications. Pandas library was used for dataset handling and preprocessing operations, while Scikit-learn was used for vectorization and similarity computation.

Movie descriptions were represented using tags containing genres, themes, and keywords. Count Vectorizer transforms textual movie descriptions into numerical vectors by constructing a vocabulary of important words.

The cosine similarity metric used in this project is mathematically represented as:

$$\text{Cosine Similarity} = \frac{A \cdot B}{\|A\| \|B\|} \quad (1)$$

where A and B represent movie feature vectors.

Movies with higher cosine similarity values are considered more closely related. The recommendation engine sorts similarity scores in descending order and selects the top five recommended movies.

The frontend application was developed using Streamlit. Users can interactively select movies from a dropdown list and generate recommendations instantly. Poster images are displayed using image URLs stored within the dataset.

4 EXPERIMENTS AND RESULTS

4.1 Experimental Setup

The recommendation system was implemented using Python programming language. Jupyter Notebook was used during experimentation and testing phases, while Streamlit was used for deployment of the interactive frontend application.

The movie dataset used in the project contains approximately 100 movies belonging to different genres including action, fantasy, science fiction, animation, and drama. Each dataset entry includes:

- Movie ID
- Movie title
- Descriptive movie tags
- Poster image URL

Count Vectorization was applied with a vocabulary size of 5000 features and English stop-word removal enabled.

4.2 Results

The developed recommendation system successfully generated relevant movie recommendations based on thematic similarity between movies.

For example, selecting movies such as “The Dark Knight” generated recommendations related to Batman and superhero themes. Similarly, science-fiction movies such as “Interstellar” generated recommendations associated with space and futuristic concepts.

The Streamlit application successfully displayed recommendations and poster images without noticeable delay. The implemented fallback image mechanism also prevented application crashes caused by invalid or missing poster URLs.

The generated recommendations demonstrated that content-based filtering can effectively recommend movies without requiring user ratings or interaction history.

5 DISCUSSION

The implemented recommendation system performed effectively for a lightweight content-based filtering application. Count Vectorization simplified textual feature extraction while cosine similarity efficiently identified relationships between movies.

One major advantage of the implemented system is that recommendations can be generated even for users with no previous interaction history. This helps address the cold start problem commonly observed in recommendation systems.

The project also demonstrates the practical application of Natural Language Processing concepts within recommendation engines. The use of descriptive tags enabled the system to identify thematic similarity among movies.

However, the project also contains certain limitations. The dataset size is relatively small compared to industrial recommendation systems used by large streaming platforms. Additionally, manually assigned tags may not fully capture deeper semantic relationships among movies.

The current implementation also does not include personalized recommendation capabilities based on user behavior or ratings. Despite these limitations, the project successfully demonstrates the implementation of recommendation system concepts using lightweight machine learning techniques.

6 CONCLUSION AND FUTURE WORK

This project presented the implementation of a content-based movie recommendation system using Natural Language Processing and cosine similarity techniques. The developed recommendation engine successfully generates movie recommendations using descriptive movie tags and thematic similarity.

The project combines machine learning concepts with

an interactive Streamlit frontend application. Experimental observations indicate that content-based filtering can generate meaningful recommendations without relying on user ratings or viewing history.

Future improvements may include increasing the dataset size, integrating hybrid recommendation techniques, and applying advanced Natural Language Processing methods such as TF-IDF or word embeddings for improved recommendation quality.

Additional enhancements may also include user authentication, recommendation personalization, and deployment using cloud-based infrastructure.

7 ARTIFACTS AND DEMONSTRATIONS

The following artifacts are associated with the project:

- **GitHub Repository:**<https://github.com/anlrudhks/My-Movie-Recommender>
- **Video Demonstration:**https://www.youtube.com/watch?v=9gSaPNtE_Rs

The repository contains source code, dataset files, frontend implementation, and supporting documentation required to reproduce the recommendation system.

8 REFERENCES

- [1] Raymond Lee, *Natural Language Processing: A Textbook with Python Implementation*, Springer Nature Singapore, 2025.
- [2] Scikit-learn Documentation, <https://scikit-learn.org>
- [3] Streamlit Documentation, <https://streamlit.io>